

JAVASCRIPT INTERVIEW QUESTIONS WITH SOLUTION

Question 1: What are the key features of JavaScript?

Solution:

- **Dynamic Typing** - You don't have to tell JavaScript what type of data a variable holds. It automatically understands the type when the code runs.
- **Prototypal Inheritance** - In JavaScript, objects can share features (like properties and methods) with other objects without using classes.
- **Functions as First-Class Citizens** - Functions are treated like regular values. You can store them in variables, pass them to other functions, or return them from functions.
- **Event-Driven** - JavaScript works based on user actions like clicking, typing, scrolling, etc. When an event happens, it can run specific code in response.
- **Asynchronous Programming** - JavaScript can do tasks like loading data from the internet without stopping other code from running. It handles time-taking tasks using callbacks, promises, or async/await.
- **DOM Manipulation** - JavaScript can change the content, structure, or style of a webpage while it's open like editing text or showing/hiding elements.

Question 2: What is the difference between var, let, and const?

Solution:

var: Declares a variable globally, or locally to an entire function regardless of block scope. Variables declared with var can be re-declared and updated

```
var x = 10;
function example() {
  var x = 20;
  console.log(x); // Output: 20
}
console.log(x); // Output: 10
```

let: Introduces block scope to variables, restricting their visibility to the block (e.g., {}) they are declared in. Variables declared with let can be updated but not re-declared in the same scope.

```
let y = 10;
if (true) {
  let y = 20;
  console.log(y); // Output: 20
}
console.log(y); // Output: 10
```

const: Also has block scope like let, but variables declared with const must be initialized with a value and cannot be reassigned. However, the properties of objects and arrays declared with const can be modified.

```
const z = 10;
// z = 20; // Error: Assignment to constant variable.

const person = { name: 'John' };
person.name = 'Jane'; // Valid, modifies the property.
```

Question 3: What is hoisting in JavaScript?

Solution: Hoisting in JavaScript refers to the behavior where variable and function declarations are moved to the top of their containing scope during the compilation phase, while the assignment remains in place.

➤ **Variable Hoisting:**

```
console.log(x); // Output: undefined
var x = 10;
```

This is interpreted as:

```
var x;
console.log(x); // Output: undefined
x = 10;
```

➤ **Function Hoisting:**

```
sayHello(); // Output: Hello!

function sayHello() {
  console.log('Hello!');
}
```

This is interpreted as:

```
function sayHello() {
  console.log('Hello!');
}

sayHello(); // Output: Hello!
```

Question 4: What are closures in JavaScript?

Solution: Closures are functions that retain access to variables from the lexical scope where they were defined, even when executed outside that scope.

```
function outerFunction() {
  let outerVar = 'I am outside!';

  function innerFunction() {
    console.log(outerVar); // Accesses outerVar from outerFunction
  }

  return innerFunction;
}

let myFunction = outerFunction();
myFunction(); // Output: I am outside!
```

Here, innerFunction forms a closure over outerVar, retaining its value even after outerFunction has finished executing.

Question 5: What is the difference between == and ===?

Solution:

- == (Equality Operator): Checks for equality after performing type coercion, meaning it converts the variables to the same type before comparison.

```
1 == '1'; // true (Number 1 is equal to String '1' after type coercion)
```

- === (Strict Equality Operator): Checks for equality without type coercion, ensuring both the value and type are the same.

```
1 === '1'; // false (Number 1 is not strictly equal to String '1')
```

Question 6: What is the difference between null and undefined?

Solution:

Feature	null	undefined
Meaning	Intentionally empty value	Variable declared but not assigned
Type	object (weird JS quirk)	undefined
Who sets it?	Developer	JavaScript engine

```
let a;  
console.log(a); // undefined (no value assigned)  
  
let b = null;  
console.log(b); // null (you gave empty value)
```

- **undefined** = "I don't know the value yet"
- **null** = "I know the value is nothing"

Question 7: What are arrow functions, and how are they different from regular functions?

Solution: **Arrow function** is a short way to write a function. Introduced in ES6.

```
// Regular function
function add(a, b) {
  return a + b;
}

// Arrow function
const add = (a, b) => a + b;
```

Feature	Regular Function	Arrow Function
Syntax	Long	Short (=>)
this	Has its own this	Uses parent function's this
Hoisting	Yes	No
Constructor?	Yes	No

```
const person = {
  name: "Vishal",
  normal: function () {
    console.log(this.name); // Vishal
  },
  arrow: () => {
    console.log(this.name); // undefined
  }
};

person.normal(); // Vishal
person.arrow(); // undefined
```

Question 8: What is the scope of a variable in JavaScript?

Solution: **Types of Scope:**

- **Global Scope** – Accessible everywhere.
- **Function Scope** – Accessible only inside the function (var).
- **Block Scope** – Accessible inside {} (with let or const).

```
let globalVar = "Global";

function test() {
  let functionVar = "Function";
  if (true) {
    let blockVar = "Block";
    console.log(blockVar); // OK
  }
  // console.log(blockVar); // ✗ Not accessible here
}
```

Question 9: What is the event loop in JavaScript? Explain call stack, callback queue, and microtask queue.

Solution: JavaScript runs code **line by line** (single-threaded). But it can also handle **asynchronous tasks** (like timers, API calls). That's where the **Event Loop** helps.

Main Parts:

- **Call Stack** – Runs normal (sync) code.
- **Web APIs** – Browser APIs handle async tasks in background (like `setTimeout`).
- **Callback Queue** – Stores async tasks to run later.
- **Microtask Queue** – Stores promises and `queueMicrotask()` (runs before callback queue).

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

console.log("4");
```

Output:

```
1
4
3
2
```

- Promise goes to Microtask Queue (runs before `setTimeout`).
- `setTimeout` goes to Callback Queue.

Question 10: What are Promises? How are they different from callbacks?

Solution: **Promise** = JavaScript object that represents a future result (success or failure).

```
const myPromise = new Promise((resolve, reject) => {
  let success = true;
  if (success) {
    resolve("Done!");
  } else {
    reject("Error!");
  }
});

myPromise
  .then(result => console.log(result)) // Done!
  .catch(error => console.log(error)); // if failed
```

Callback vs Promise:

Feature	Callback	Promise
Syntax	Nested, hard to read	Clean and chainable
Error Handling	Must handle manually	Has <code>.catch()</code>
Modern?	Old way	Modern way (ES6)

➤ Callback Hell:

```
doTask1(() => {  
  doTask2(() => {  
    doTask3(() => {  
      console.log("Done");  
    });  
  });  
});
```

➤ Using Promises:

```
doTask1()  
  .then(doTask2)  
  .then(doTask3)  
  .then(() => console.log("Done"))  
  .catch(err => console.log(err));
```

Question 11: What is async/await in JavaScript? How does it improve asynchronous code?

Solution:

async/await is a cleaner way to write code that waits for something to finish (like loading data).

- async makes a function and return a **Promise**.
- await tells JavaScript to **wait** until a Promise is done.

```
function getData() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Data is ready!");  
    }, 2000);  
  });  
}  
  
async function showData() {  
  console.log("Start");  
  const result = await getData(); // waits here  
  console.log(result); // shows after 2 seconds  
  console.log("End");  
}  
  
showData();
```

Output:

```
Start
(wait 2 sec)
Data is ready!
End
```

Question 12: What is the difference between synchronous and asynchronous code?

Solution:

Type	Meaning	Example
Synchronous	Code runs line by line, one at a time	Cooking step-by-step
Asynchronous	Code can pause and continue later	Set a timer, then do something else

```
// Synchronous
console.log("1");
console.log("2");
```

```
// Asynchronous
console.log("1");
setTimeout(() => {
  console.log("2");
}, 1000);
console.log("3");
```

Output:

```
1
3
2
```

Question 13: What is the DOM? How is it manipulated using JavaScript?

Solution: DOM = **Document Object Model**. It is a **tree-like structure** of your HTML page that JavaScript can access and change.

Common DOM Methods:

➤ getElementById, querySelector, innerText, innerHTML, style

```
<h2 id="heading">Welcome</h2>
<button onclick="changeText()">Click</button>

<script>
  function changeText() {
    document.getElementById("heading").innerText = "Hello Vishal!";
  }
</script>
```

After clicking the button, the heading changes = **DOM Manipulation**

Question 14: What is event delegation in JavaScript?

Solution: Instead of adding click to each item, we add **one click to the parent**.

```
<ul id="menu">
  <li>Home</li>
  <li>About</li>
</ul>

<script>
  document.getElementById("menu").addEventListener("click", function(e) {
    if (e.target.tagName === "LI") {
      alert("You clicked: " + e.target.innerText);
    }
  });
</script>
```

You can click any li, even new ones added with JS - no need for multiple listeners, any clicked will show an alert.

Question 15: What are higher-order functions? Give examples.

Solution: A **Higher-Order Function (HOF)** is a function that: **Takes another function** as an argument **or Returns another function**. JavaScript treats functions as **values**, so you can pass/return them like variables.

Example 1: Takes a function

```
function greet(name) {
  return "Hello " + name;
}

function welcome(callback) {
  console.log(callback("Vishal"));
}

welcome(greet); // Hello Vishal
```

Example 2: Returns a function

```
function multiply(x) {
  return function(y) {
    return x * y;
  };
}

let double = multiply(2);
console.log(double(5)); // 10
```

Question 16: What are pure functions and side effects in JavaScript?

Solution:

- **Pure Function:** A function that **always gives the same output** for the same input. It does **not change anything outside** the function (no side effects).

Example of Pure Function:

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3)); // 5
```

- **Side Effects of Function:** When a function **changes something outside**, like the DOM, a variable, or console.

Example with Side Effect:

```
let count = 0;  
  
function increase() {  
  count++; // changes outside variable  
  console.log(count);  
}  
  
increase(); // Output: 1
```

Question 17: What is the difference between map (), filter (), and reduce ()?

Solution:

Method	What it does	Returns
<code>map()</code>	Transforms each item	New array
<code>filter()</code>	Keeps items that match a condition	New array
<code>reduce()</code>	Combines all items into one value	Single value

- **map() Example:**

```
let numbers = [1, 2, 3];  
let doubled = numbers.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```

- **filter() Example:**

```
let numbers = [1, 2, 3, 4];  
let even = numbers.filter(num => num % 2 === 0);  
console.log(even); // [2, 4]
```

- **reduce() Example:**

```
let numbers = [1, 2, 3, 4];  
let sum = numbers.reduce((acc, num) => acc + num, 0);  
console.log(sum); // 10
```

Question 18: What is the difference between shallow copy and deep copy?

Solution:

➤ **Shallow Copy:** Only copies the **first layer** of an object. Nested objects are **still connected**

```
let original = { name: "Vishal", address: { city: "Delhi" } };
let copy = { ...original };

copy.address.city = "Mumbai";
console.log(original.address.city); // Mumbai (changed)
```

➤ **Deep Copy:** Copies **everything**, including nested objects. No connection with original

```
let original = { name: "Vishal", address: { city: "Delhi" } };
let deepCopy = JSON.parse(JSON.stringify(original));

deepCopy.address.city = "Mumbai";
console.log(original.address.city); // Delhi (unchanged)
```

Question 19: What is the difference between call (), apply (), and bind ()?

Solution: All three (call, apply, and bind) are used to **change the value of this** in a function — i.e., they tell a function **who it should act like it belongs to**.

Method	Executes function?	Argument Format	Returns
<code>call()</code>	Yes	Arguments passed one-by-one	Result
<code>apply()</code>	Yes	Arguments passed as an array	Result
<code>bind()</code>	✗ No (gives back a new function)	Arguments one-by-one or later	New function

Simple Example for All Three:

```
const person = {
  name: "Vishal"
};

function greet(greeting, age) {
  console.log(`${greeting}, I'm ${this.name} and I'm ${age} years old.`);
}
```

call()

```
greet.call(person, "Hello", 22);
// Output: Hello, I'm Vishal and I'm 22 years old.
```

It calls the function immediately, passing arguments one by one.

apply()

```
greet.apply(person, ["Hi", 22]);  
// Output: Hi, I'm Vishal and I'm 22 years old.
```

Same as call(), but arguments in an array.

bind()

```
const greetVishal = greet.bind(person, "Hey", 22);  
greetVishal();  
// Output: Hey, I'm Vishal and I'm 22 years old.
```

It doesn't run the function immediately, but gives back a new function which you can call later.

Question 20: What are JavaScript data types? Explain primitive vs reference types.

Solution:

- **Primitive Types (store value directly):** These are simple values — not objects, and they don't have properties or methods. Like - (Number, String, Boolean, Null, Undefined, Symbol, BigInt)

```
let name = "Vishal"; // string  
let age = 22; // number
```

- **Reference Types (store address/reference):** These are complex data types — they are objects and stored as references in memory also they have properties or methods. Like - (Object, Array, Function)

```
let person = { name: "Vishal" }; // object  
let numbers = [1, 2, 3]; // array
```

Question 21: What are IIFEs (Immediately Invoked Function Expressions)?

Solution: **IIFE** means: a function that runs immediately after it is defined.

Syntax:

```
(function () {  
  console.log("I ran immediately!");  
})();
```

We use IIFE because:

- To **avoid polluting the global scope** (like creating global variables).
- Useful when you want some code to **run only once** (setup code, etc).

Example:

```
(function () {  
  let secret = "hidden data";  
  console.log("This is IIFE");  
})();  
  
console.log(secret); // ✗ Error: secret is not defined
```

"secret" is not accessible outside of IIFE block, or can say it is a block scope using "let".

Question 22: What is a prototype? What is prototypal inheritance?

Solution:

- **Prototype:** Every JavaScript object has a hidden property called `__proto__` that points to its prototype (parent object).
- **Prototypal Inheritance:** It allows one object to use properties/methods from another object.

Example:

```
let person = {
  greet() {
    console.log("Hello!");
  }
};

let student = {
  name: "Vishal"
};

student.__proto__ = person;

student.greet(); // ✓ Hello!
```

Here, student is using greet() method from person via prototype.

Question 23: What is the difference between class and prototype-based inheritance?

Solution:

Feature	Class-based (ES6+)	Prototype-based (Old JS style)
Syntax	Uses <code>class</code> keyword	Uses functions and <code>__proto__</code>
Inheritance	<code>extends</code> keyword	Manual <code>__proto__</code> or <code>Object.create()</code>
Readability	Cleaner and modern	Less readable
Behind the scenes	Uses prototype internally	Direct prototype usage

Class Example:

```
class Person {
  sayHi() {
    console.log("Hi");
  }
}

class Student extends Person {}

let s = new Student();
s.sayHi(); // Hi
```

Question 24: What is the use of "this" keyword in JavaScript?

Solution: **"this"** refers to the **current context or object** depending on **how** the function is called.

Examples:

➤ Global scope:

```
console.log(this); // In browser, it refers to window object
```

➤ Inside object:

```
let user = {  
  name: "Vishal",  
  sayHello() {  
    console.log(this.name);  
  }  
};  
  
user.sayHello(); // Vishal ✓
```

➤ Standalone function:

```
function test() {  
  console.log(this);  
}  
  
test(); // In strict mode: undefined, otherwise: window
```

Question 25: How does "this" behave in arrow functions vs regular functions?

Solution: JavaScript's this keyword can behave differently based on **how a function is defined and called**.

- **Regular Functions** → this depends on **how** the function is called. Use **regular functions** when you need your function to have its **own this** (like in methods).
- **Arrow Functions** → this is **lexically bound**, i.e., it uses this from the **surrounding scope**. Also, Arrow functions **do not have their own this**. They take the value of this from the **surrounding (parent) function or block**.

Feature	Regular Function	Arrow Function
Own <code>this</code> binding	✓ Yes (dynamic)	✗ No (inherits from parent)
Use case	Object methods, constructor functions	Callbacks, inner functions, <code>setTimeout</code> , etc.
In object method	<code>this</code> refers to the object	<code>this</code> refers to outside (usually window/global)
In callback	May lose <code>this</code>	Safely inherits <code>this</code> from parent

Examples:

1. Regular Function — this depends on the caller

```
const user = {  
  name: "Vishal",  
  greet: function () {  
    console.log(this.name);  
  }  
};
```

```
user.greet(); // Output: Vishal ✓
```

In this case: “this” refers to user object, because user is calling the greet method.

2. Arrow Function — this is lexically scoped

```
const user = {  
  name: "Vishal",  
  greet: () => {  
    console.log(this.name);  
  }  
};
```

```
user.greet(); // Output: undefined ✗
```

In this case: Arrow function doesn't bind its own this. So this.name comes from outside (probably global scope), where name is undefined.

Side-by-side Comparison both Arrow and Regular Function:

```
const person = {  
  name: "Vishal",  
  
  regularFunc: function () {  
    console.log("Regular:", this.name);  
  },  
  
  arrowFunc: () => {  
    console.log("Arrow:", this.name);  
  }  
};  
  
person.regularFunc(); // Regular: Vishal ✓  
person.arrowFunc();   // Arrow: undefined ✗
```

- “regularFunc” works because it gets this from the person object.
- “arrowFunc” fails because it gets this from outside the object, where this.name is undefined.

Real Use Case of “this” for Regular and Arrow Function: setTimeout()

- Regular Function loses this inside setTimeout:

```
const obj = {
  name: "Vishal",
  sayHi: function () {
    setTimeout(function () {
      console.log(this.name); // ✗ undefined (here, this = window/global)
    }, 1000);
  }
};

obj.sayHi();
```

- Arrow Function fixes this:

```
const obj = {
  name: "Vishal",
  sayHi: function () {
    setTimeout(() => {
      console.log(this.name); // ✔ Vishal
    }, 1000);
  }
};

obj.sayHi();
```

The arrow function uses “this” from the sayHi method context.

Question 26: What are modules in JavaScript? Difference between CommonJS and ES6 modules?

Solution: Modules are used to **split JavaScript code into separate files**, so that each file handles a specific part of your app.

They help with:

- Code reusability
- Better file organization
- Clean architecture

➤ CommonJS (Used in Node.js)

- Uses require() to import
- Uses module.exports to export
- Runs **synchronously**

```
// math.js
const add = (a, b) => a + b;
module.exports = add;

// main.js
const add = require('./math');
console.log(add(2, 3)); // 5
```

➤ ES6 Modules (Used in Browsers & Modern JS)

- Uses import and export keywords
- Runs **asynchronously**

```
// math.js
export const add = (a, b) => a + b;

// main.js
import { add } from './math.js';
console.log(add(2, 3)); // 5
```

Feature	CommonJS	ES6 Modules
Syntax	<code>require</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Environment	Node.js	Browsers & Modern JS
Loading	Synchronous	Asynchronous

Question 27: What is the difference between localStorage, sessionStorage, and cookies?

Solution:

Feature	localStorage	sessionStorage	cookies
Expiry	Never (until deleted)	On tab close	Based on expiry time
Size limit	~5MB	~5MB	~4KB
Sent with requests	✗ No	✗ No	✓ Yes

1. localStorage

- Data is stored **permanently** (until manually cleared)
- Size limit: ~5MB
- Only accessible from JavaScript on the same domain

```
localStorage.setItem('name', 'Vishal');
console.log(localStorage.getItem('name')); // Vishal
```

2. sessionStorage

- Data is stored **only for the session**
- It cleared when tab or browser is closed

```
sessionStorage.setItem('token', '123abc');
```

3. Cookies

- Small data (4KB), stored with **expiry date**
- Automatically sent with every HTTP request

```
document.cookie = "user=Vishal; expires=Fri, 31 Dec 2025 23:59:59 GMT";
```

Question 28: What is a debounce function? What is throttling?

Solution: Both are used to **limit the number of times a function is called**, often in performance-heavy events like typing, scrolling, resizing, etc.

➤ **Debounce:** Waits for a pause in activity before executing the function.

Use Case: Search input box (When user stopped typing the function get executed, not every time when user type each letter)

```
function debounce(func, delay) {
  let timer;
  return function () {
    clearTimeout(timer);
    timer = setTimeout(func, delay);
  };
}
```

➤ **Throttle:** Ensures the function runs **once every X milliseconds**, no matter how often the event occurs.

Use Case: Window scroll event

```
function throttle(func, delay) {
  let lastTime = 0;
  return function () {
    const now = new Date().getTime();
    if (now - lastTime >= delay) {
      func();
      lastTime = now;
    }
  };
}
```

Question 29: What is the difference between setTimeout () and setInterval ()?

Solution:

➤ **setTimeout ():** Runs a function **once** after a delay (in ms).

```
setTimeout(() => {
  console.log("Hello after 2 sec");
}, 2000);
```

➤ **setInterval ():** Runs a function **repeatedly** at a fixed time interval.

```
setInterval(() => {
  console.log("Repeats every 1 sec");
}, 1000);
```

Feature	setTimeout()	setInterval()
Runs	Once after delay	Repeatedly after interval
Clear	clearTimeout(id)	clearInterval(id)

Question 30: What are memory leaks? How can you prevent them?

Solution: A memory leak happens when **your program keeps using memory but doesn't release it**, even if it's no longer needed. This slows down performance or even crashes your app.

Common Causes:

1. Global variables not cleared
2. Event listeners not removed
3. DOM elements not deleted properly
4. Timers (setInterval) not cleared

How to Prevent Memory Leaks?

1. Use let and const properly
2. Remove unused event listeners
3. Use clearInterval() or clearTimeout()
4. Avoid keeping unnecessary references to DOM
5. Use developer tools (Chrome DevTools → Memory tab) to track usage

Example:

```
const btn = document.getElementById("clickBtn");

function handleClick() {
  console.log("Clicked!");
}

btn.addEventListener("click", handleClick);

// ✅ Good practice:
btn.removeEventListener("click", handleClick);
```